AI 454: NLP

Implementation assignment 1

Due: April 4th , 2026

11:59PM

General instructions

Language: use python. You may use libraries such as NumPy and gensim, but core components (e.g., cosine similarity and analogy computation) must be implemented manually.

Teams of Two (Mandatory): This assignment must be completed in teams of exactly two people.

Collaboration Requirement: Teams are expected to discuss every detail together, rather than dividing the workload. Both team members should be actively involved in all aspects of the assignment, from data processing and model implementation to reporting results, thus ready to answer any question.

Code: Make it self-contained and include clear run instructions. Submit via e-learning, with team members' names in the code and report.

Documentation: **DON'T** add documentation to your code at all (should be free of documentation), code with documentation will not be discussed.

Content: Answer all questions in the report.

Grade: This assignment is graded out of **40** points, which will be scaled to 10 points (**10 % of the overall course grade**) Your final grade will consider the code, report, and, importantly, the quality of group discussion. Missing discussion on any required point will result in losing weight for that point (this includes the ability to answer any question related to the code). If a group fails to discuss three or more points, they risk losing the entire assignment grade —so put in your best effort!

Plagiarism Warning: Copying code from online resources or using AI tools **(e.g., ChatGPT)** to generate solutions will result **in a grade of zero.** Students are expected to implement the assignment independently to demonstrate their understanding. Ensure that all submitted code and written answers are your own work.

Late Submission Penalty: You may submit your solution up to one day later. A penalty of 20% will be applied to your grade for late submissions. No submissions will be accepted beyond this one-day extension.

Rasha Obeidat

Part 1 — Intrinsic Evaluation of Word Embeddings: Glove vs word2vec(skip-gram) of size 300

In this part of the assignment, you will work on exploring pre-trained word embeddings and designing word analogy tests. You are expected to implement components that load pre-trained **Word2Vec**, **GloVe** and **fastText** embeddings and then use these embeddings to test both syntactic and semantic word analogies.

Step1: Loading Pre-trained Embeddings:

1. **Word2Vec**: Load a pre-trained Word2Vec (skip-gram) model using an appropriate Python library (e.g., gensim).
2. **GloVe**: Load pre-trained GloVe embeddings. Since GloVe files are usually in text format, convert them to the Word2Vec format if necessary (using available conversion scripts).
3. **FastText:** Load pre-trained FastText embeddings (300-dimensional) using a suitable library (e.g., gensim). FastText models include subword (character n-gram) information, which helps handle rare and morphologically rich words more effectively.

Step2: **Designing Your Own Word Analogy Tests**

- Create **10 semantic analogy tests** and **10 syntactic analogy tests (one theme for each)**.
- **Semantic Analogies:** For example, tests like "man : king :: woman : ?" that check for conceptual relationships.
- **Syntactic Analogies:** For example, tests like "walk : walking :: run : running" that explore grammatical transformations.
- **Originality:** Do not use existing tests from any datasets; create original analogies on your own. Ensure that your tests are uniquely created and not generated by tools like ChatGPT.
- **Be creative in creating your analogy, trivial ones won't receive the full specified mark**

Implementation Details:

- Use vector arithmetic to compute analogies and implement the cosine similarity function manually. Use the standard analogy formulation:

  vector(b) - vector(a) + vector(c)

- Use Python with basic packages like NumPy and gensim; avoid high-level ML libraries.
- If you encounter compatibility issues with gensim—such as binary incompatibility errors due to version mismatches with NumPy—you may implement an alternative approach. Instead of relying on gensim, you can load the embeddings directly from

text files (e.g., from GloVe or a converted Word2Vec file) by reading the file line by line into a Python dictionary. This custom loader approach bypasses gensim and often resolves these compatibility problems. Details and a sample implementation for this alternative method have been provided.

Organizing the Results:

- **Test Tables:** Organize your 10 semantic tests and 10 syntactic tests in two tables (with clear captions). Each table should have the following columns:
  - **Test #:** The test number.
  - **Test:** The analogy format (e.g., "singer : music :: painter : ?").
  - **Expected Answer:** The correct answer you expect.
  - **GloVe Result:** fill it based on the results of your code
  - **Word2Vec Result:** fill it based on the results of your code
  - **FastText Result:** fill it based on the results of your code
- **Example Table (Semantic Analogy Tests) (4 points):**

Test #	**Test**	**Expected Answer**	**GloVe 300 Result**	**Word2Vec 300 Result**	**FastText 300 Result**
1	singer : music :: painter : ?	art			
2	...	...			
...	...	...			
10	...	...			

- **Example Table (syntactic Analogy Tests) (4 points):**

Test #	**Test**	**Expected Answer**	**GloVe 300 Result**	**Word2Vec 300 Result**	**FastText 300 Result**
1	quick: quickly:: careful: ?	carefully			

2	...	...			
...	...	...			
10	...	...			

Step 3: Accuracy Computation

Compute the accuracy for each test set using both:

- Top-1 accuracy
- Top-5 accuracy

This is done by checking whether the correct answer appears among the top predicted candidates.

Top-1 accuracy measures whether the correct answer is the highest-ranked prediction.

$$\text{Top-1 Accuracy} = \frac{\text{Number of correct predictions}}{10} \times 100\%$$

Top-5 accuracy measures whether the correct answer appears within the top 5 predicted candidates.

$$\text{Top-5 Accuracy} = \frac{\text{Number of correct predictions within top 5}}{10} \times 100\%$$

Summarize the overall results in a separate table with empty cells for the placeholders. For example:

Analogy Type	**GloVe 300 Accuracy**	**Word2Vec 300 Accuracy**	**fastText 300**
Semantic			
Syntactic			

Analogy Top-1 Accuracy **(3 points):**

Analogy Type	**GloVe 300 Accuracy**	**Word2Vec 300 Accuracy**	**fastText 300**

Semantic			
Syntactic			

Analogy Top-5 Accuracy **(3 points):**

Part 2 — Intrinsic Evaluation of Word Embeddings: Glove 100 vs Glove 300 vs Glove 50

Use the same analogy examples you created in part 1 to test Glove 300 (already done in part 1, but you need to copy what you have got) vs. Glove of embedding size 100 and 50 and repeat the analogy request and compute and report the top-1 and top-5 accuracy

- **Example Table (semantic Analogy Tests) (3 points):**:

Test #	**Test**	**Expected Answer**	**GloVe 50 Result**	**GloVe 100 Result**	**Glove 300 results** **(just copy them from part 1)**
1	singer : music :: painter : ?	art			
2	...	...			
...	...	...			
10	...	...			

- **Example Table (syntactic Analogy Tests) (3 points):**

Test #	**Test**	**Expected Answer**	**GloVe 50 Result**	**GloVe 100 Result**	**Glove 300 results**
1	quick: quickly:: careful: ?	carefully			
2	...	...			

...	...	...			
10	...	...			

Analogy Type	**GloVe 50 Accuracy**	**GloVe 100 Accuracy**	**Glove 300 Accuracy**
Semantic			
Syntactic			

Analogy Top-1 Accuracy **(3 points):**

Analogy Type	**GloVe 50 Accuracy**	**GloVe 100 Accuracy**	**Glove 300 Accuracy**
Semantic			
Syntactic			

Analogy Top-5 Accuracy **(3 points):**

Step 4 (5 points): Analysis and Discussion

- **Discussion: Word2Vec vs. GloVe vs. FastText**

  Based on your results obtained from the **semantic** and **syntactic** analogy tests, discuss the performance of the three models (**Word2Vec**, **GloVe**, and **FastText**) together. In your discussion:

  - Compare the **accuracy** of the three models on **semantic analogy tasks**.
  - Compare the **accuracy** of the three models on **syntactic analogy tasks**.
  - Clearly explain any **similarities or differences** you observe in their performance.
  - Relate your observations to the **underlying training mechanisms** of each model:
    - **Word2Vec:** predictive model using **local context**.
    - **GloVe:** factorizes a **global co-occurrence matrix**.
    - **FastText:** incorporates **subword (character n-gram) information**, improving handling of **morphology and rare words**.

- Based on these mechanisms, explaining **why** certain models perform better in specific tasks (e.g., syntactic vs. semantic, common vs. rare words, morphologically rich forms) your analogies should support this discussion.
- Summarize key **strengths** and **weaknesses** of each model.

- **Effect of Embedding Dimension in GloVe**
  - Compare the results of **GloVe-300** , **GloVe-50** and **GloVe-100**.
  - Discuss what you observe regarding:
  - representation quality,
  - analogy accuracy,
  - trade-off between **dimensionality**, **computational cost**, and **performance**.
- Your grade for this part will depend on how well your reported results align with your analysis, and on your ability to clearly articulate and justify the same observations and conclusions during the in-class discussion.

Part 3 — viualization of word embedding using Glove 100 (6 points)

Create **three sets of related words**, where each set contains **at least 6 semantically related terms** (you must design your own sets).

Tasks:

1. Obtain the **GloVe (100d)** and **Word2Vec (100d)** embeddings for all selected words.
2. Use **t-SNE** to reduce the embeddings to **2D**.
3. Visualize the words on a 2D plot (label each word and distinguish sets using colors).
4. Analyze the results:
   - Do related words cluster together?
   - Are the sets well separated?
5. Compare GloVe vs Word2Vec:
   - Which model produces better clustering?
   - Justify your answer based on your observations.

Final Note: There will be **3 points** on the originality and clarity (consistent font type, size, sectioning, table legends) of your code and report, and compliance with the assignment guidelines.